ISTA 130

Function Rules and Jargon

What’s new?

Turtle graphics and functions - parameters and arguments in more depth.

D2L

Remember extra credit is available in the form of worksheets! They’re posted in the Weekly modules and are all due the day before the Midterm (for the worksheets prior to the Midterm exam).

Last time

- ➔ Turtle graphics
- ➔ Objects and methods
- ➔ Variables
- ➔ Modules
- ➔ Common Mistakes

Construct a brand new turtle named ‘tallulah’:

```
tallulah = turtle.Turtle()
```

`turtle.Turtle():` this tells Python to construct (make) a turtle. But we need to give the turtle a name so we can tell it to do things for us. How do we do that?

`=`: we use **`=`** to give the turtle the name **`tallulah`**. We call this ***assignment***. Think “**`tallulah`** gets assigned a turtle”.

`tallulah`: **`tallulah`** is a ***variable*** - a name that refers to a value, but the value can change - hence the name “variable”.

= vs ==

In Python, = doesn't mean "equals"; it means "gets assigned".

So what does mean "equals"? == of course!

> Just kidding about the of course part. Don't worry about this, yet, just remember that = is how we connect a variable name to an object (like a turtle, a number, a string, etc.)

Objects and Types

Recall that everything in Python is an object.

Every object has a ***type***. **`tallulah`** is a turtle object. **`3`** is an **`int`** (Python for ‘integer’). So is `777`. Objects of the same type have the same properties. In other words, they hold the same kind of data, have the same methods, work the same when they are passed to functions, and the same operators work on them in the same way (e.g., +).

Python Strings

A ***string*** is a sequence of characters.

Python stores text in programs in strings.

Python's string data type is **`str`** (Python for string). Every string in Python is of type `str`.

Variables vs Literals

Data in programs can be stored in variables, or it can be explicitly specified using literals.

Examples:

- `x = 3`
  - `x` is an integer variable, `3` is an integer literal.
- `my_str = "I heart Freya!"`
  - `my_str` is a string variable, `"I heart Freya!"` is a string literal.

This is the mantra: ***Variables refer to values, literals are values.***

turtle.pencolor(<color_string>)

We can change the color of a turtle's pen:

```
>>> snoopy.pencolor(blue)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'blue' is not defined
>>> snoopy.pencolor('blue')
>>> snoopy.forward(50)
```

HTML color names.

Let's add some color!

Notes on the template file (`template.py`)

At the top we have a documentation comment.

Next we import turtle (and other things, soon!)

Next we have a comment telling us where to put functions, which we will practice this week.

`def main():` this is where we will put our main program.; As we have seen, it is actually in the form of a function, which I will explain later in the course.

Problems we need to solve:

Turns out our program is really long. In our Week 2 discussion we drew a square. What if we wanted to draw more squares? Lots more code.

What if later in another program we want to draw a square? Death by a thousand copy-and-pastes.

Functions

If we think about drawing a square, we realize that it is an algorithm - a list of instructions for solving a problem or performing a task.

We would like to write reusable code to implement this algorithm that we can use in other programs as a one-line instruction.

We call these new instructions ***functions***.

Functions

Here is the general form of a function definition in Python:

```
def <function_name>():
    <instruction 1>
    <instruction 2>
    ...

    return None
```

Python uses indentation to tell what code belongs to the function.

Remember to adjust tab settings in your editor!

Function Rules and Jargon

This is called the function ***signature/header***

```
def draw_L(param_turtle):
    param_turtle.left(90)
    param_turtle.forward(25)


def main():
    tallulah = turtle.Turtle()
    draw_L(tallulah)
```

Function Rules and Jargon

This is called the function ***function body***

```
def draw_L(param_turtle):
    param_turtle.left(90)
    param_turtle.forward(25)


def main():
    tallulah = turtle.Turtle()
    draw_L(tallulah)
```

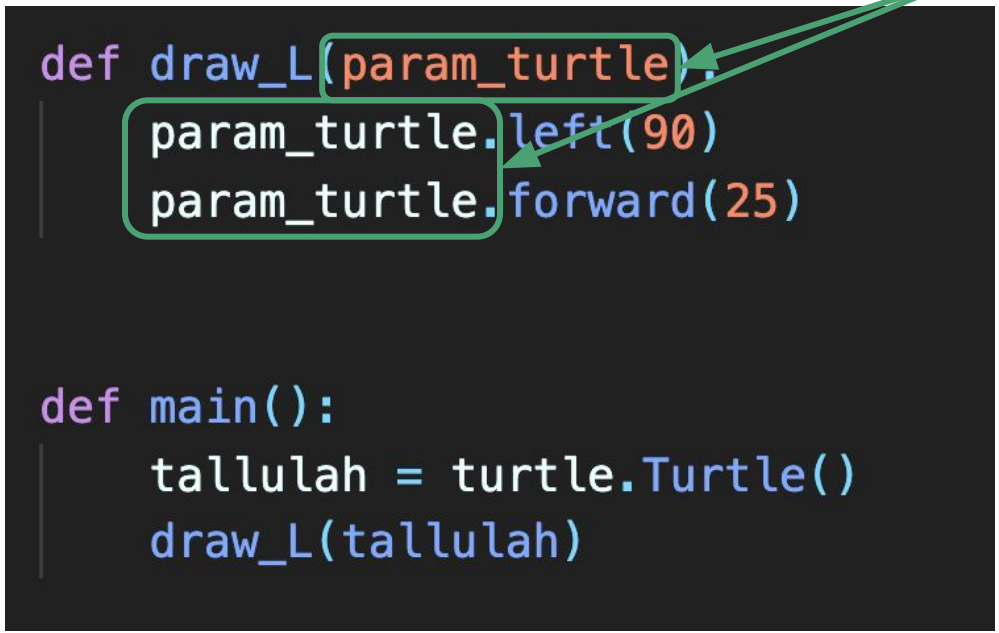
Function Format

The function body is indented under the function header.

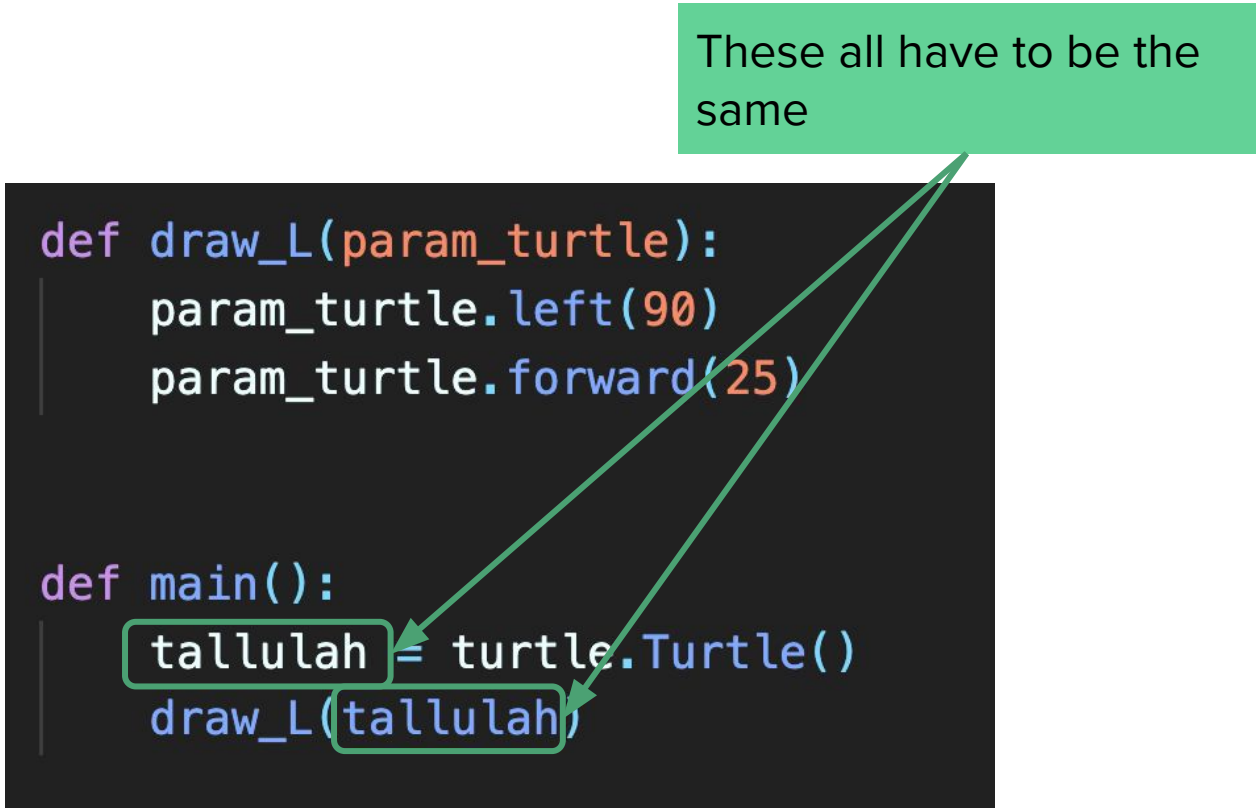
This is how Python knows which code belongs to the function.

Function Rules and Jargon

These all have to be the same



Function Rules and Jargon



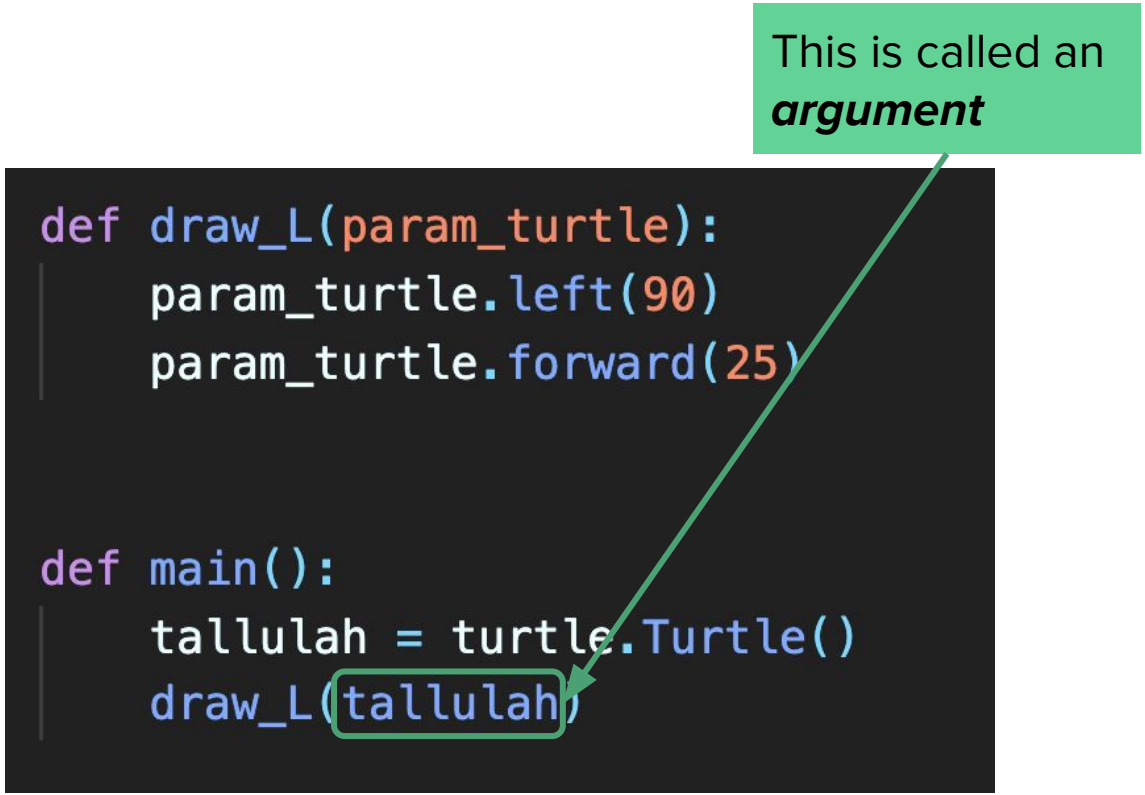
Function Rules and Jargon

This is called a ***parameter***

```
def draw_L(param_turtle):
    param_turtle.left(90)
    param_turtle.forward(25)


def main():
    tallulah = turtle.Turtle()
    draw_L(tallulah)
```

Function Rules and Jargon



Function Rules and Jargon

This is called a
function definition

```
def draw_L(param_turtle):
    param_turtle.left(90)
    param_turtle.forward(25)


def main():
    tallulah = turtle.Turtle()
    draw_L(tallulah)
```

Function Rules and Jargon

This is called a ***function call***

```
def draw_L(param_turtle):
    param_turtle.left(90)
    param_turtle.forward(25)


def main():
    tallulah = turtle.Turtle()
    draw_L(tallulah)
```

Parameters vs Arguments

Parameters are variables listed in the function signature for use in the function body. Python will get the values for the parameters from the function calls.

Arguments are variables or literals in function calls that are passed to the function to give the parameters values.

Arguments in function calls correspond to parameters in function definitions.

This is the mantra: ***Parameters are in function definitions, arguments are in function calls.***

Hw1: What NOT To Do

- ➔ Do not construct turtles inside your functions.
- ➔ In other words, the function definition should not contain the code `turtle.Turtle()`
- ➔ This would make your code horribly inflexible.

Hw1: What To Do

➔ To solve this problem, we have to be able to tell our function what we want it to use. We will use the programming equivalent of a pronoun.

Definition of **pronoun** (n)

Bing Dictionary

pro·noun [prṓ nòwn]

1. word replacing noun: a word that substitutes for a noun or a noun phrase

“it”

- ➔ Meaningless without knowing what object is referring to.
- ➔ We know the meaning from the context.
- ➔ The pronoun temporarily stands for some object.
- ➔ Meaningless if used out of context.

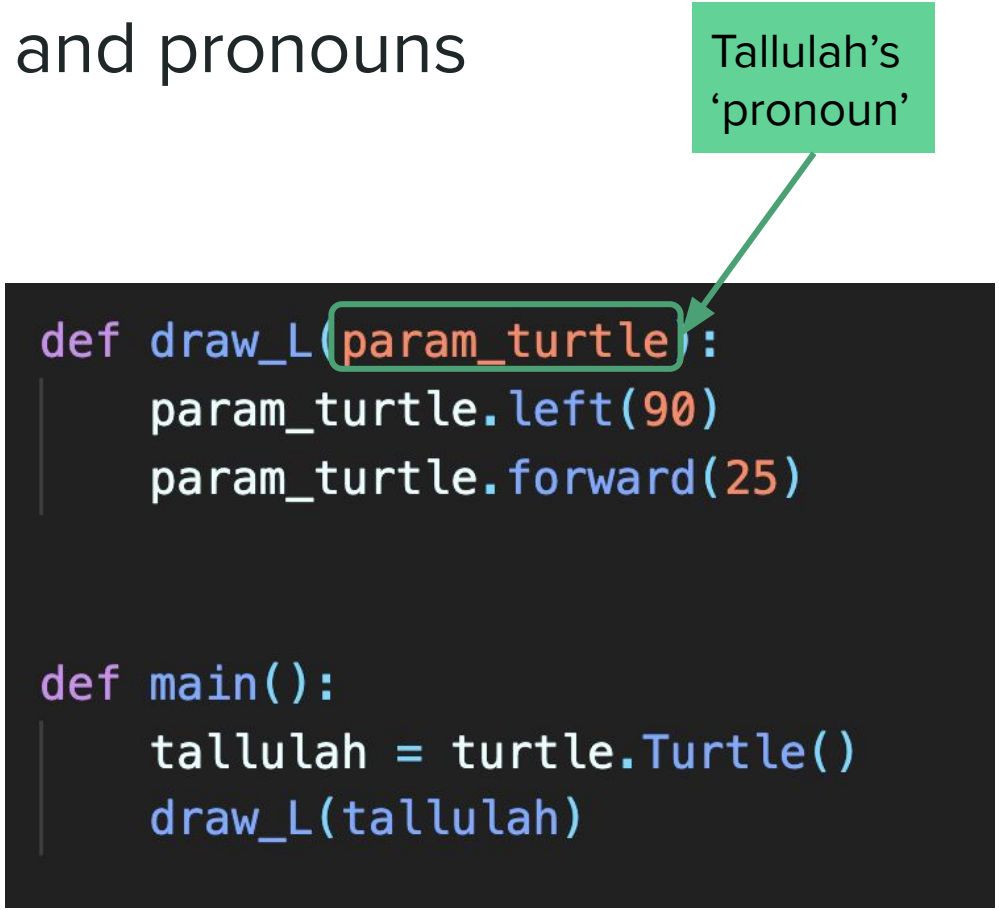
How does this solve our problem?

When we define our function, we will use a pronoun (whose name we will choose) in the function instructions. We call these pronouns ***parameters***.

When we call the function, we will tell the function what object to substitute for our pronoun.

This is what we're doing in our coding examples.

Parameters and pronouns



Defining Functions with Parameters

A function definition containing the parameter **`it`**.

The parameter behaves like a pronoun.

```
def square(it):
    it.right(90)
    it.forward(100)
    it.right(90)
    it.forward(100)
    it.right(90)
    it.forward(100)
    it.right(90)
    it.forward(100)
    it.right(90)
    return None
```

Calling functions that have parameters in their definition

```
tallulah = turtle.Turtle()
square(tallulah)
```

- ➔ **`square(tallulah)`** tells Python to execute the instructions in the function **`square`**, substituting **`tallulah`** for the parameter ‘it’ everywhere in the code.
- ➔ When we pass an object to a function, in this case **`tallulah`**, it is called an argument.

Parameters vs Arguments

Parameters are found in the function signature and body. They are placeholders, like pronouns. They are a special kind of variable.

Arguments are found in function calls. They are values, either variables referring to values or literals explicitly specifying values.

Parameter Names

Parameter names should tell us what kind of object they refer to.

In other words, **`it`** is bad, **`a_turtle`** is good.

```
def square(a_turtle):
    a_turtle.right(90)
    a_turtle.forward(100)
    a_turtle.right(90)
    a_turtle.forward(100)
    a_turtle.right(90)
    a_turtle.forward(100)
    a_turtle.right(90)
    a_turtle.forward(100)
    a_turtle.right(90)
    return None
```

More on names...

- ➔ Good: **`triangle`**, **`a_turtle`**, **`atom_count`**, ...
- ➔ Bad: **`12days`**, **`-worms`**, ...
- ➔ Ugly: **`q34`**, **`brp_rb`**, **`xyz`**, ...

Begin names with letter, separate words with underscores, use lowercase letters and numbers.

Pick meaningful names.

Multiple parameters

Suppose I want to be able to draw squares of different sizes, but I don't want to have to change my code every time.

The size of my square is determined by how far I tell my turtle to move:

```
def square(a_turtle):
    a_turtle.right(90)
    a_turtle.forward(100)
    a_turtle.right(90)
    a_turtle.forward(100)
    a_turtle.right(90)
    a_turtle.forward(100)
    a_turtle.right(90)
    a_turtle.forward(100)
    a_turtle.right(90)
    return None
```

Multiple parameters

Suppose I want to be able to draw squares of different sizes, but I don't want to have to change my code every time.

The size of my square is determined by how far I tell my turtle to move:

```
tallulah.forward(100)
    # perhaps
tallulah.forward(50)
    # or maybe
tallulah.forward(7)
```

Multiple parameters

If I do not want the size of my square to be predetermined, I need to pass **`forward()`** an argument that is a variable:

```
tallulah.forward(side_len)
```

And I need to be able to tell **`square(a_turtle)`** how big **`side_len`** should be. So I add another parameter:

```
def square(a_turtle, side_len):
    a_turtle.forward(side_len)
    ...
```

Multiple parameters

A call to my redefined function:

```
square(tallulah, 100)
```

How does Python know which argument goes with which parameter? It assumes the arguments are passed in the same order the parameters are defined (i.e., it matches them by position).

Some Turtle methods

You could do hw1 with just these methods:

```
forward()
right()
penup()
pendown()
pensize()
pencolor()
```

Turtle Graphics Help

https://docs.python.org/3.3/library/turtle.html?highlight=turtle

Office Hours

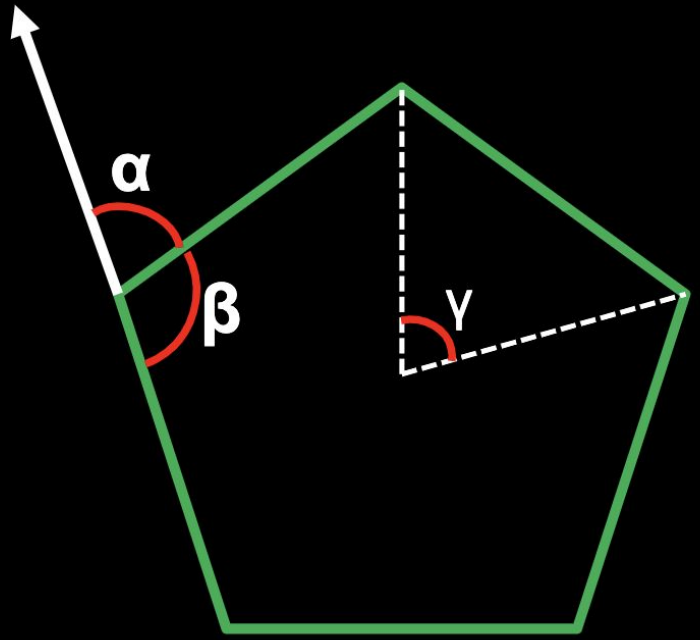
Section Leaders

Discussion Group	Name	Email	Office Hours
Group 1	Sara Cielaszyk	scielaszyk@email.arizona.edu	Th 1-3pm
Group 2	Shaylynn Basham	shaylynnwalsh@email.arizona.edu	Th 7-9pm
Group 3	Erika Kirkpatrick	elk1@email.arizona.edu	W 3-4pm, Th 3-4pm
Group 4	Nick Tang*	nicholastang@email.arizona.edu	Th 4-6pm
Group 5	Quan Le	minhquanle@email.arizona.edu	W 6-7pm, Th 11am-12pm

Section Leader Office Hours will be held via Zoom | * indicates Head Section Leader

* you can find your group by looking in D2L Classlist

The turn angle α to draw a polygon with n sides



$$\alpha = 180 - \beta$$
$$= \gamma$$
$$= 360 / n$$

To Do

Watch Functions, Types, Math Lecture

Watch Coding example: Let's Draw a House

Quiz 2 - Function Jargon

Functions Discussion

Hw1